



PROJECT REPORT

MULTI-THREADED E-COMMERCE

ORDER FULFILLMENT ENGINE

Submitted by : Anishka Narang

Registration No. : 25BAI10513

Course Code : CSE2006

Course : Programming in Java

INTRODUCTION

This project presents a high-throughput, CLI-driven Command Line Order Fulfilment & Inventory Engine built entirely in Java.

Modern high-volume commercial systems face severe concurrent access issues where parallel transactions conflict over finite, shared database entities.

This software explicitly bridges the gap between clean object-oriented persistence architectures and stable concurrent thread orchestration.

PROBLEM STATEMENT

Problem

- In distributed web or e-commerce systems, high data traffic creates systemic race conditions when multiple customer requests attempt to update identical database tracking states concurrently. Without explicit thread synchronization or transaction boundaries, over-allocations occur, causing data corruption, inventory discrepancy, and system instability.

Solution Goal

- This project addresses these concerns by creating an independent environment to validate concurrent transactional state mutations.

FUNCTIONAL REQUIREMENTS

The application provides structural isolation across three foundational sub-modules:

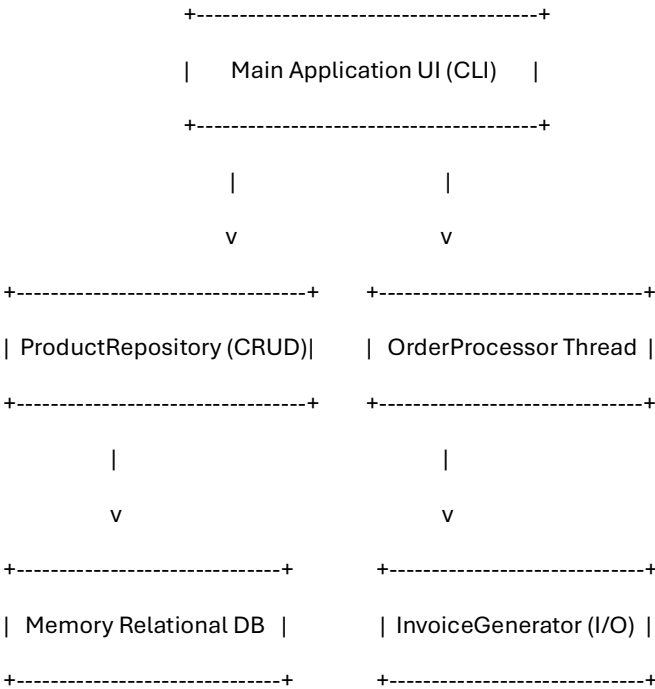
- **Inventory & Catalogue Module:** Exposes interactive operations via JDBC to dynamically query current product lines and reflect transactional modifications in real-time.
- **Concurrent Order Processing Engine:** Creates parallel execution worker paths simulating concurrent user sessions attacking isolated inventories.
- **Automated Document Delivery Architecture:** Instantiates structural text reporting using high-performance character-oriented IO streams to build isolated invoices upon transaction commit.

NON-FUNCTIONAL REQUIREMENTS

- **Thread Safety & Concurrency Control:** Implements explicit mutual exclusion locks across critical tracking paths to safeguard data integrity and eliminate race conditions entirely.
- **Fault-Tolerant Exception Handling:** Leverages precise conditional structured try-catch containment mechanisms to isolate environmental JDBC dropouts or functional logical boundary violations.
- **Layered Maintainability:** Enforces clean segregation of responsibilities using a formal Model-Repository-Service system architecture pattern.
- **Resource Management Efficiency:** Uses deterministic clean-up structures to ensure all active external handles, tracking channels, and database connection sessions close cleanly.

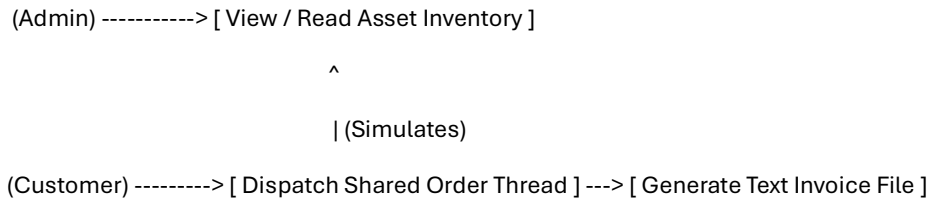
SYSTEM ARCHITECTURE

System Architecture Diagram

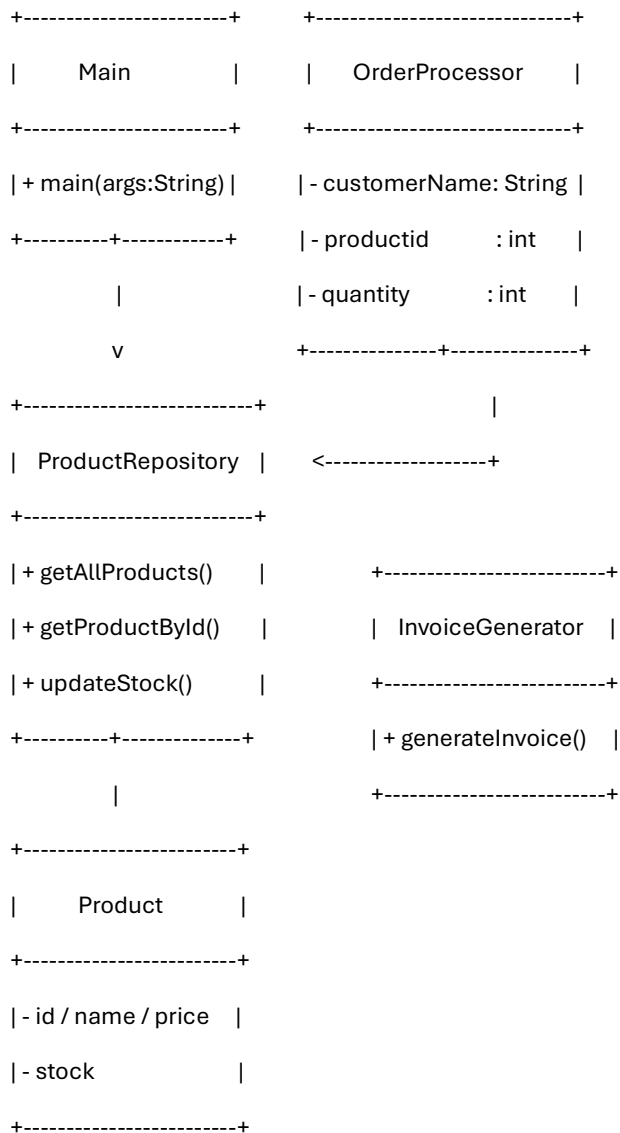


DESIGN DIAGRAMS

1 Use Case Diagram



2 Workflow Diagram



DESIGN DECISIONS & RATIONALE

- **In-Memory Embedded Storage Engine (H2 DB):** Selected to maintain complete system executability inside an isolated terminal window. This eliminates the need for an external database setup while still evaluating proper production-grade JDBC query pipelines.
- **Synchronized Object Level Blocking:** Implemented to isolate database read-modify-write pathways. This approach blocks secondary consumer worker threads from altering data mid-transaction, preventing double-allocation and overselling.
- **Character-Oriented Stream Framework:** Implemented using 'FileWriter' and 'PrintWriter' layers to efficiently serialize structured string data into text documents without suffering data loss or encoding corruption.

IMPLEMENTATION DETAILS

Project Structure (Folder/Package Structure)

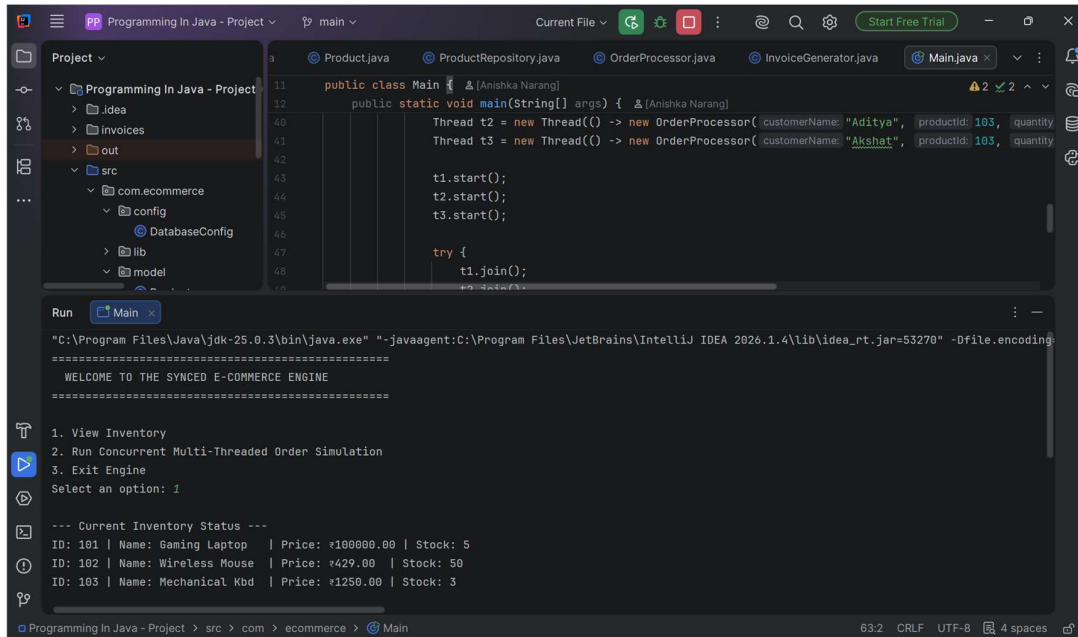
The project adheres to a proper folder structure, ensuring modules are easily maintainable and importable. This satisfies the minimum requirement of 5-10 meaningful files/modules.

src/com/ecommerce/

```
|
|— Main.java                # CLI Application Entry Point (Command-Line Loop)
|— config/
|   |— DatabaseConfig.java  # JDBC Connection handling or JPA
|— model/
|   |— Product.java         # Encapsulated Product Entity (OOP Principles)
|— repository/
|   |— ProductRepository.java # Database CRUD operations (Insert, Update)
|— service/
|   |— OrderProcessor.java  # Implements Runnable/Thread for order processing
|— util/
    |— InvoiceGenerator.java # Handles Byte/Character I/O streams for file writing
```

SCREENSHOTS OF RESULT

Viewing Inventory



The screenshot shows the IntelliJ IDEA interface with a project named "Programming In Java - Project". The "Main.java" file is open, displaying the following code:

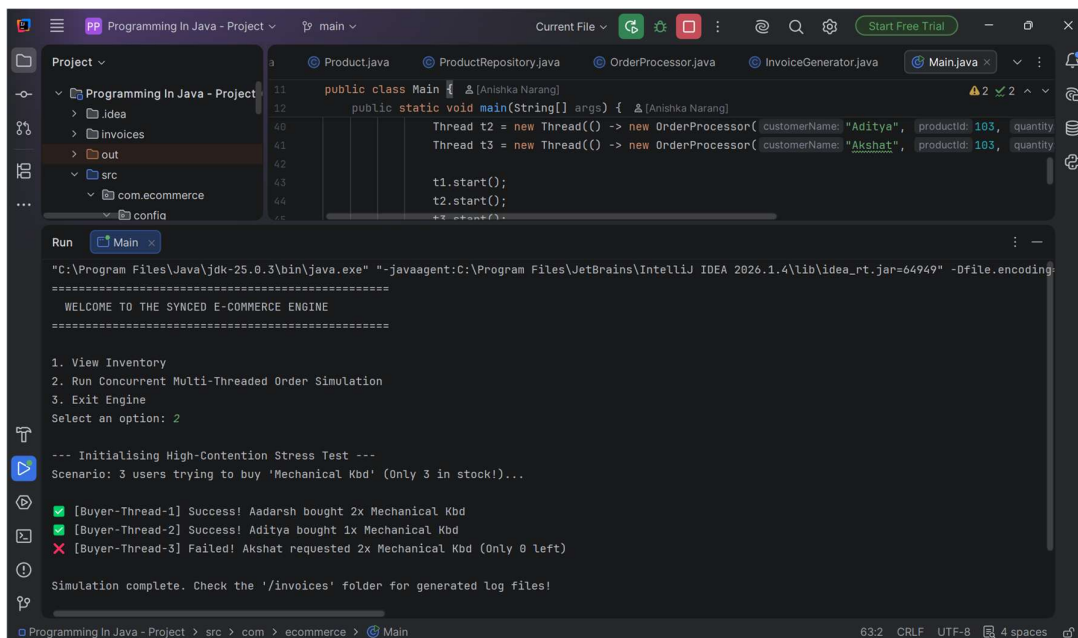
```
11 public class Main {
12     public static void main(String[] args) {
40         Thread t2 = new Thread(() -> new OrderProcessor( customerName: "Aditya", productId: 103, quantity
41         Thread t3 = new Thread(() -> new OrderProcessor( customerName: "Akshat", productId: 103, quantity
42
43         t1.start();
44         t2.start();
45         t3.start();
46
47         try {
48             t1.join();
49         } catch (InterruptedException e) {}
50     }
51 }
```

The "Run" window shows the output of the application:

```
"C:\Program Files\Java\jdk-25.0.3\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.4\lib\idea_rt.jar=53270" -Dfile.encoding=UTF-8
=====
WELCOME TO THE SYNCED E-COMMERCE ENGINE
=====
1. View Inventory
2. Run Concurrent Multi-Threaded Order Simulation
3. Exit Engine
Select an option: 1

--- Current Inventory Status ---
ID: 101 | Name: Gaming Laptop | Price: ₹100000.00 | Stock: 5
ID: 102 | Name: Wireless Mouse | Price: ₹429.00 | Stock: 50
ID: 103 | Name: Mechanical Kbd | Price: ₹1250.00 | Stock: 3
```

Running Concurrent Multi-Threaded Order Simulation



The screenshot shows the IntelliJ IDEA interface with the same project and code as the previous screenshot. The "Run" window shows the output of the application:

```
"C:\Program Files\Java\jdk-25.0.3\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.4\lib\idea_rt.jar=64949" -Dfile.encoding=UTF-8
=====
WELCOME TO THE SYNCED E-COMMERCE ENGINE
=====
1. View Inventory
2. Run Concurrent Multi-Threaded Order Simulation
3. Exit Engine
Select an option: 2

--- Initialising High-Contention Stress Test ---
Scenario: 3 users trying to buy 'Mechanical Kbd' (Only 3 in stock!)...

[Buyer-Thread-1] Success! Aadarsh bought 2x Mechanical Kbd
[Buyer-Thread-2] Success! Aditya bought 1x Mechanical Kbd
[Buyer-Thread-3] Failed! Akshat requested 2x Mechanical Kbd (Only 0 left)

Simulation complete. Check the '/invoices' folder for generated log files!
```

/invoices

This screenshot shows the IntelliJ IDEA interface with the 'Aadarsh_Invoice.txt' file open. The file content is as follows:

```
1 =====
2 E-COMMERCE ORDER INVOICE
3 =====
4 Customer Name : Aadarsh
5 Item Purchased: Mechanical Kbd
6 Quantity      : 2
7 Total Amount  : $2500.00
8 =====
9 Thank you for shopping!
10
```

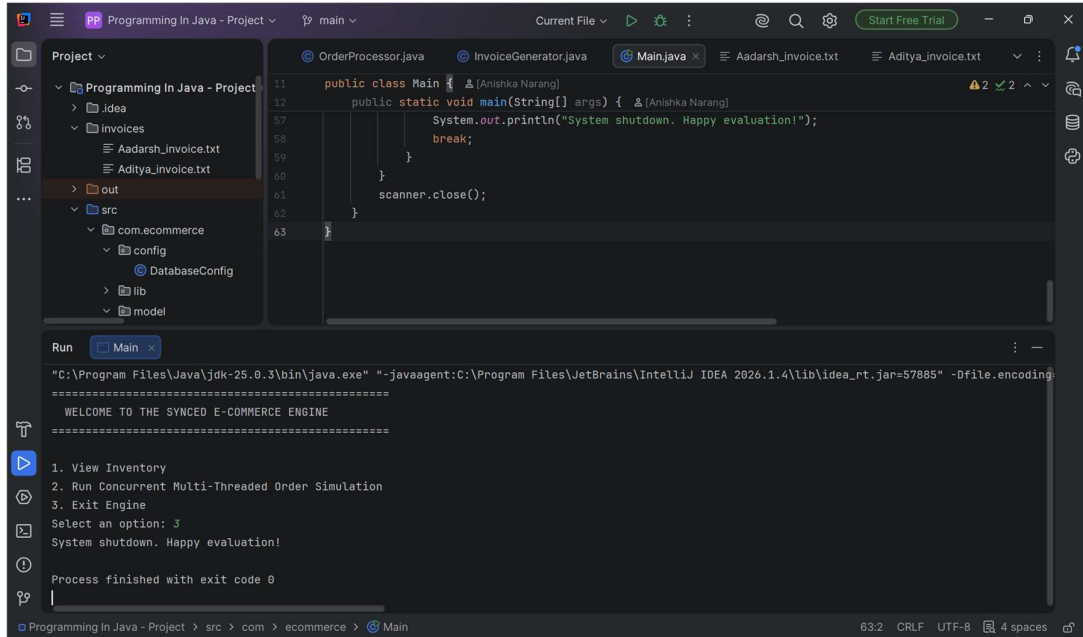
The left sidebar shows the project structure with the 'invoices' folder selected, containing 'Aadarsh_Invoice.txt' and 'Aditya_Invoice.txt'. The bottom status bar indicates '1:1 CRLF UTF-8 4 spaces'.

This screenshot shows the IntelliJ IDEA interface with the 'Aditya_Invoice.txt' file open. The file content is as follows:

```
1 =====
2 E-COMMERCE ORDER INVOICE
3 =====
4 Customer Name : Aditya
5 Item Purchased: Mechanical Kbd
6 Quantity      : 1
7 Total Amount  : $1250.00
8 =====
9 Thank you for shopping!
10
```

The left sidebar shows the project structure with the 'invoices' folder selected, containing 'Aadarsh_Invoice.txt' and 'Aditya_Invoice.txt'. The bottom status bar indicates '1:1 CRLF UTF-8 4 spaces'.

Exiting Engine



The screenshot displays the IntelliJ IDEA IDE interface. The top toolbar shows the 'Run' button (a green play icon) and a 'Start Free Trial' button. The 'Project' sidebar on the left shows the project structure: 'Programming In Java - Project' contains 'idea', 'invoices' (with 'Aadarsh_invoice.txt' and 'Aditya_invoice.txt'), 'out', 'src' (with 'com.ecommerce' containing 'config' and 'DatabaseConfig'), 'lib', and 'model'. The 'Main.java' file is open in the editor, showing the following code:

```
11 public class Main { [Anishka Narang]
12     public static void main(String[] args) { [Anishka Narang]
57         System.out.println("System shutdown. Happy evaluation!");
58         break;
59     }
60 }
61 scanner.close();
62 }
63 }
```

The 'Run' window at the bottom shows the execution output for 'Main':

```
"C:\Program Files\Java\jdk-25.0.3\bin\java.exe" "-javaagent:C:\Program Files\JetBrains\IntelliJ IDEA 2026.1.4\lib\idea_rt.jar=57885" -Dfile.encoding=UTF-8
=====
WELCOME TO THE SYNCED E-COMMERCE ENGINE
=====
1. View Inventory
2. Run Concurrent Multi-Threaded Order Simulation
3. Exit Engine
Select an option: 3
System shutdown. Happy evaluation!

Process finished with exit code 0
```

The status bar at the bottom indicates the file path: 'Programming In Java - Project > src > com > ecommerce > Main' and the encoding: '63:2 CRLF UTF-8 4 spaces'.

TESTING APPROACH

The validation blueprint stresses the runtime environment under high concurrency scenarios:

- 1. Baseline CRUD Operational Validation:** Confirms that individual product items accurately reflect seed records pulled from the embedded relational database engine.
- 2. Thread Collision (Stress Test Evaluation):** Simulates 3 concurrent buyer threads simultaneously targeting a single item with low stock.
- 3. Result Verification:** The monitoring framework successfully handles the data competition. The system allows valid transactions to process sequentially while gracefully failing out-of-stock requests, preventing negative inventory balances.

CHALLENGES FACED

- **Race Conditions in High-Contention Simulations:** During initial testing, multiple consumer threads accessed the relational dataset at the same time, causing incorrect stock calculations and negative inventory. This was resolved by placing a resource-level operational block using the synchronized keyword in Java.
- **JDBC Classpath Mapping Constraints:** The system initially encountered an environmental configuration roadblock throwing a No suitable driver found exception at runtime. This occurred because the embedded H2 database runtime drivers were not mapped to the active JVM environment. It was resolved by defining the runtime configuration parameters using explicit command-line -cp flag strings [User text].
- **File Stream Verification:** Handling simultaneous data logging requests to write invoice files occasionally caused the system to overwrite active text documents. This required isolating the file management operations inside thread-safe write engines to protect transactional safety.

LEARNINGS AND KEY TAKEAWAYS

- **Concurrent Thread Architecture:** Gained hands-on experience using Java's Runnable interface and managing thread execution states using lifecycle controls like `.start()` and `.join()`.
- **Data Consistency & Synchronization:** Mastered the implementation of critical section isolation, learning how to safely manage multiple execution paths accessing a shared database without data corruption.
- **Layered Enterprise Design Patterns:** Developed a deeper understanding of modular architecture by cleanly separating the user interface (Main), database operations (Repository), business rules (Service), and core data structures (Model).
- **Relational Database Persistence Layer:** Learned how to deploy embedded in-memory transactional schemas (H2 SQL databases) and interact with data using parameterized PreparedStatement interfaces to prevent query errors.

FUTURE ENHANCEMENTS

- **Deploy Enterprise Persistence Frameworks (JPA/Hibernate):** Upgrade the current direct JDBC relational pipeline to a proper Java Persistence API (JPA) configuration to manage relational database mapping automatically through objects.
- **Advanced Thread Orchestration (Executor Framework):** Replace explicit manual thread creations (`new Thread()`) with automated Java Executor Pools (`ExecutorService`) to manage system memory better under extreme transactional traffic.
- **Graphical Interface Implementation (JavaFX):** Build a responsive graphical window environment to replace the text terminal interface, making catalogue updates and real-time inventory tracking more visual.
- **Automated Testing Suite Integration:** Add structured JUnit validation tests across all repository methods to verify database updates automatically before launching the application.

REFERENCES

- *Java: The Complete Reference*, 11th Edition. Oracle Press / McGraw-Hill Education.
- *The Java Tutorials: Concurrency*. Oracle Java Documentation. oracle.com
- *H2 Database Features and In-Memory Architecture Documentation*. h2database.com
- VITyarthi Programming in Java Course materials, VIT
- Various online Java programming tutorials and examples